

Voltgrid Engine Assessment

Heliform Analytics

2026-06-17

Table of contents

Introduction	4
What Voltgrid Does	4
Document Map	4
 Core modeling engine	 6
Documentation of the current core engine	7
Processing	7
 1.b Optimization Engine	 8
Overview	8
General Schema	10
Notation	11
Step 2 — Cooperative Tradeoff	11
Base problem — shared by both phases	11
Phase 1 vs Phase 2	12
Phase 1 — LP	12
Phase 2 — QP	13
Summary	14
 1.c Supply Profiles and Add-ons	 15
Supply File Format	15
What is a Flow?	15
Supply as a Constraint in the Optimization	16
Supply as a Decision Variable (Supply Optimizer)	16
Decision Variables	17
Constraints	17
Objective Function	17
Pipeline: Sources → Flows → Constraints	18
 1.d Minimal Supply	 19
 1.e Supply Optimization	 20
 Appendix — Code Reference	 21
A.1 Optimization Engine	21
Step 1 — Problem Construction	21
Step 2 — Cooperative Tradeoff	23

A.2 Supply Math to Code	25
Part 1 — Supply as a Constraint (<code>cooperative_tradeoff</code>)	25
Part 2 — Supply as a Decision Variable (<code>supply_optimizer.py</code>)	25
Assessing the validity of current core engine	28
Validation Using Measurement Datasets	28
Sensitivity Analysis	28
Robustness Plan for Calls to Optimization Solver	28
Securing IP while improving current engine	29
Long-term R&D vision	30
R&D use cases assessment	31
Sponsors and KOL strategy	32
Change and integration	33
Usage ramp-up	34
Datasets	35

Introduction

What Voltgrid Does

The goal of Voltgrid is to **simulate a distributed energy grid at steady state**: given a supply profile (available primary energy) and a current grid composition (generator capacities), find the power-dispatch state the network converges to at equilibrium.

At equilibrium, two quantities fully describe the grid:

- **Dispatch rates** μ_i per generator — which nodes are active and at what rate they deliver power, consistent with their installed capacities.
- **Power flows** v_x^i per line per generator — what each node draws and exports, including transfer flows exchanged between nodes and power exported to the external load.

Together these define a complete operating portrait of the grid: not only *which nodes are present* (capacity) but *what they are doing* (dispatch) — which flows are produced, consumed, and ultimately available to the load.

i Why steady state?

The optimization does not model dynamics over time. It finds the flow distribution that satisfies all balance constraints simultaneously — the state the network would reach if given sufficient time to settle on the provided supply profile.

Document Map

1. Technical documentation of the engine

- 1.a Processing — pipeline steps before the optimization starts
- 1.b Understanding the optimization engine
- 1.c Supply profiles and add-ons
- 1.d Minimal supply
- 1.e Supply optimization

2. Assessing the validity of current methodology

- 2.a Validation using measurement datasets

- 2.b Sensitivity analysis
- 2.c Robustness plan for calls to optimization solver

3. **Securing IP while improving current engine**

- 3.a Using training to correct errors

Core modeling engine

Documentation of the current core engine

Processing

Pipeline steps that occur before the optimization solver is called.

 Question for the engine team

Which processing steps make sense to describe here? To what extent might they affect the definition of our optimization problems?

1.b Optimization Engine

Notation follows the [GridFlow engine reference](#). Code variable names are given in parentheses.

Overview

Voltgrid splits the optimization into two successive steps.

Step 1 — Problem Setup assembles the LP structure: nodes are loaded, balance constraints ($S \cdot v = 0$) are encoded, and individual and network dispatch objectives are defined. No solver is called at this stage.

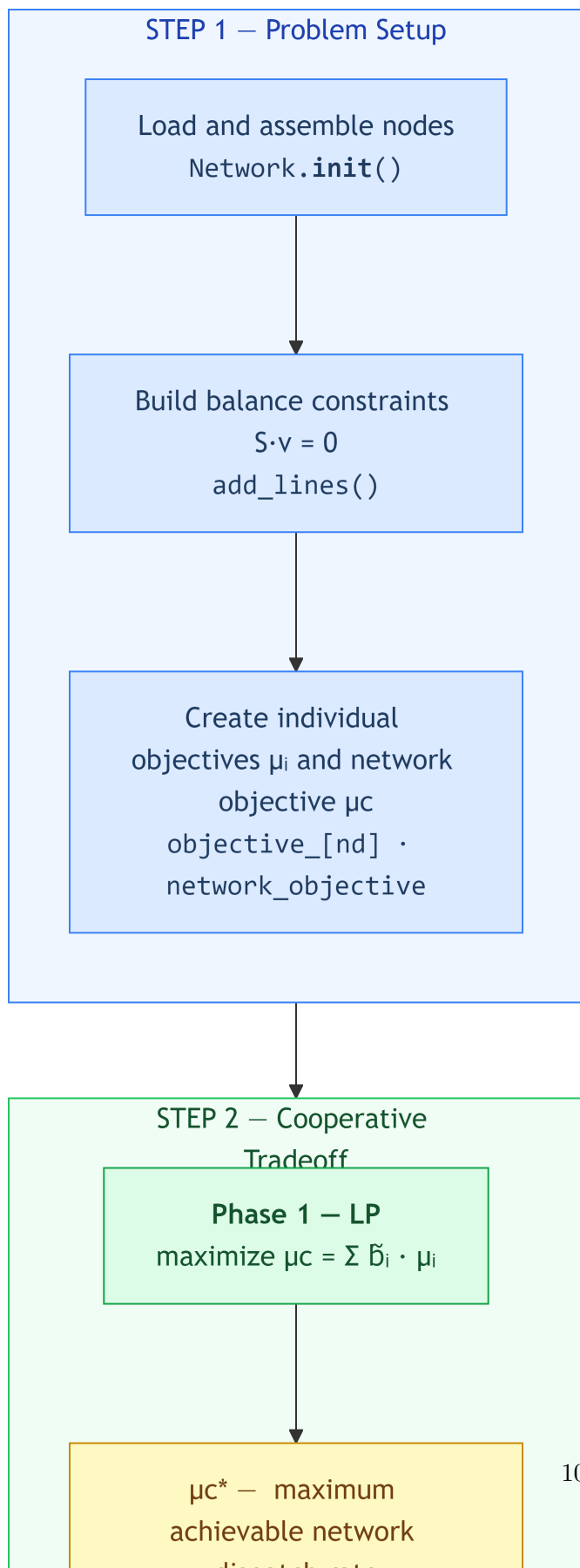
Step 2 — Cooperative Tradeoff then solves the problem in two phases. The need for two phases comes from a well-known failure mode of classic dispatch applied to networks: a single highly efficient node can monopolize all available supply and drive every other member to zero output. To prevent this, GridFlow introduces a tradeoff parameter $\alpha \in [0, 1]$ that relaxes the network dispatch constraint, giving lower-output nodes room to remain active.

Concretely:

1. **Phase 1 (LP)** — run a standard dispatch to obtain the maximum achievable network dispatch rate μ_c^* .
2. **Phase 2 (QP)** — fix $\mu_c \geq \alpha \cdot \mu_c^*$ and minimize $\sum_i \mu_i^2$, which distributes dispatch as evenly as possible across all nodes.

This approach is much faster than multi-objective alternatives and yields an optimum conceptually similar to fair-share dispatch: each node operates at its maximum rate without diminishing the output of others — hence *cooperative*. In the absence of additional constraints, the resulting dispatch rates are correlated with node capacity within a network, not as a prior assumption but as a direct consequence of the optimization.

General Schema



Notation

Symbol	Meaning	Code variable
v_x^i	Flow of line x in node i	<code>v_forward</code> , <code>v_reverse</code> per line
μ_i	Dispatch rate of node i	<code>objective_[nd]</code> (constraint primal value)
μ_c	Network dispatch rate $= \sum_i \tilde{b}_i \cdot \mu_i$	<code>network_objective</code>
μ_c^*	Maximum achievable network dispatch rate	<code>min_dispatch</code> (output of Phase 1)
$\tilde{b}_i$	Relative capacity of node i : $b_i / \sum_j b_j$	<code>row.capacity</code>
α	Tradeoff parameter $\in [0, 1]$	<code>fraction</code>
S	Incidence matrix	encoded by GridCore via <code>add_lines()</code>

Step 2 — Cooperative Tradeoff

The LP assembled in Step 1 is solved in two successive phases. Phase 2 **replaces the objective in place** (LP $\rightarrow$ QP) before calling the solver again — the constraint structure is not rebuilt.

i Why two phases?

Phase 1 finds the theoretical maximum dispatch rate μ_c^* . Phase 2 then asks: *given that the network dispatches at least at $\alpha \cdot \mu_c^*$, what is the most equitable distribution of dispatch rates across nodes?* The two-phase structure avoids solving a bilevel problem directly.

Base problem — shared by both phases

Decision variables

Variable	Description
v_x^i	Flow of each line in each node (all nodes combined)
μ_i (<code>objective_[nd]</code>)	Individual dispatch rate — stored as a constraint, read as primal value

Variable	Description
μ_c (<code>network_objective</code>)	Network dispatch rate, linked by equality to $\sum_i \tilde{b}_i \cdot \mu_i$

Constraints

Constraint	Formulation
Balance	$S \cdot v = 0$ for every bus in every node
Non-negative dispatch	$\mu_i \geq 0$ for every node i
Transfer flow bounds	import bound $/B$, export bound unchanged (supply-controlled)
Capacity coupling	Stoichiometric coefficient on shared bus $= \tilde{b}_i$
Network objective	$\mu_c = \sum_i \tilde{b}_i \cdot \mu_i$ (equality constraint)

 $S \cdot v = 0$ is never explicit

The incidence matrix S is never built as a matrix object. It is encoded implicitly by GridCore: each bus becomes a constraint, and each line writes its incidence coefficients into that constraint via `set_linear_coefficients()`.

Phase 1 vs Phase 2

Phase 1 — LP

Objective

$$\max \quad \mu_c = \sum_i \tilde{b}_i \cdot \mu_i$$

Additional constraint (*optional — only if `min_dispatch` is specified*)

$$\mu_i \geq \mu_i^{\min} \quad \forall i$$

Solver call

```
# problems.py : 78-82
net.objective = net.scale * net.variables.network_objective
min_dispatch = optimize_with_retry(net, ...) / net.scale
```

Output $\rightarrow \mu_c^*$: maximum achievable network dispatch rate (`min_dispatch`)

💡 This is a standard LP

Phase 1 is a plain linear program. Any solver works (GLPK, CPLEX, Gurobi). The result μ_c^* is used only as a bound in Phase 2 — it is not the final answer.

Phase 2 — QP

Objective

$$\min \sum_i \mu_i^2$$

Minimizing the sum of squares penalizes solutions where one node monopolizes dispatch. It finds the dispatch rate distribution closest to equal sharing across nodes.

Additional constraints

$$\alpha \cdot \mu_c^* \leq \mu_c \leq \mu_c^*$$

where α (fraction) is the tradeoff parameter set by the user.

Objective construction in code

```
# problems.py : 53-66
def regularize_l2_norm(network, min_dispatch):
    l2 = Zero
    for nd in network.nodes:
        node_obj = network.constraints["objective_" + nd]
        ex = sum(v for v in node_obj.variables if (v.ub - v.lb) > 1e-6)
        if not isinstance(ex, int):
            l2 += (network.scale * (ex**2)).expand()
    network.objective = -l2 # minimize L2 maximize -L2
```

Output → `NetworkSolution` : individual dispatch rate μ_i per node

⚠ Requires a QP-capable solver

Phase 2 introduces a quadratic objective. GLPK cannot solve it. Gurobi, CPLEX, or the hybrid solver are required.

Summary

Phase	Objective	Key added constraint	Solver required	Output
1 — LP	maximize $\mu_c = \sum_i \tilde{b}_i \cdot \mu_i$	$\mu_i \geq \mu_i^{\min}$ (optional)	Any (LP)	μ_c^* (scalar)
2 — QP	minimize $\sum_i \mu_i^2$	$\alpha \cdot \mu_c^* \leq \mu_c \leq \mu_c^*$	QP-capable (Gurobi / CPLEX)	NetworkSolution (μ_i per node)

1.c Supply Profiles and Add-ons

How supply inputs and reserve injections are incorporated as constraints on transfer flows.

Supply File Format

In the pipeline, a supply profile is passed to GridFlow as a **packed artifact** (zipped archive). Inside this archive, the core data is a **CSV file** with two essential columns:

Column	Description
id	Transfer line identifier (e.g. <code>EX_pv__D_e</code> for solar import)
flux	Maximum import flow for that source ($kW \cdot h^{-1} \cdot MVA^{-1}$)

Each row represents one source available in the grid environment. The `id` maps directly to a transfer line in the network model.

What is a Flow?

A **flow** (v_i) is the rate at which power is drawn or produced by a transfer line. In the context of a supply profile, it represents the **maximum quantity of power that nodes can import per unit time and per unit of installed capacity**.

The unit $kW \cdot h^{-1} \cdot MVA^{-1}$ breaks down as:

- kW : amount of power (kilowatts)
- h^{-1} : per hour
- MVA^{-1} : normalized by total installed capacity

This normalization by MVA is key: the flow value is **independent of the site's footprint**. It expresses a rate relative to the network itself, not to the external load.

The conversion from raw supply to flow is:

$$v_i = \frac{D_i \times (1 - A_i) \times 1000}{Mw_i \times B_{total}}$$

Symbol	Description	Unit
v_i	Maximum flow available to nodes	$kW \cdot h^{-1} \cdot MVA^{-1}$
D_i	Raw supply of source i	$kW \cdot h^{-1}$
A_i	Upstream losses (before the bus)	unitless
Mw_i	Conversion factor of source i	$kW \cdot unit^{-1}$
B_{total}	Total installed capacity	MVA

Supply as a Constraint in the Optimization

Inside `cooperative_tradeoff`, the supply profile is **not a decision variable** — it is a **constraint** on import flows, applied before any optimization begins (`grow.py:55`):

```
net.medium = medium[medium.index.isin(ex_ids)]
```

This sets an upper bound on each transfer line:

$$0 \leq v_{\text{import},i} \leq v_i$$

Nodes cannot draw more than what the supply allows. This is the operating ceiling imposed on the optimization problem.

The supply profile therefore contributes to both phases:

- **Phase 1 (LP)**: limits the feasible space for maximizing network dispatch
- **Phase 2 (QP)**: same bounds apply when redistributing flows under the tradeoff constraint

Supply as a Decision Variable (Supply Optimizer)

In `supply_optimizer.py`, the logic is reversed: the goal is to find **which supply profile to schedule** in order to reach an operating objective (e.g. maximize reserve power output).

Variable	Description	Bounds
----------	-------------	--------

Decision Variables

Variable	Description	Bounds
f_i	Amount of source i (units)	$[f_i^{\min}, f_i^{\max}]$
a_i	Addition relative to current profile	$[0, f_i^{\max} - f_i^{\text{current}}]$
r_i	Reduction relative to current profile	$[0, f_i^{\text{current}} - f_i^{\min}]$
$\delta_i \in \{0, 1\}$	Binary indicator: was source i modified?	—

Constraints

Constraint	Expression	Purpose
Source balance	$f_i = f_i^{\text{current}} + a_i - r_i$	Tracks net change per source
Max sources changed	$\sum_i \delta_i \leq N$	Limits schedule complexity
Change indicator	$a_i + r_i \leq 1000 \cdot \delta_i$	Links binary to actual change
Load envelope	$\text{load}_{\min} \leq \sum_i f_i \cdot \text{kW}_i \leq \text{load}_{\max}$	Operating feasibility
FBA — balance (<i>optional</i>)	$\sum_r S_{m,r} \cdot v_r = 0$ for each bus m	Incidence feasibility of grid operation
FBA — supply coupling (<i>optional</i>)	$v_{EX_i} \geq \frac{\text{supply_flux}_i}{\text{FLUX_CONVERSION_FACTOR}}$	Node draw cannot exceed what the supply provides

i FBA constraints change the nature of the problem

Without `--model`: the objective operates on supply flows directly — the optimizer reasons about source quantities and their power content.

With `--model`: the balance constraints are injected via `_embed_network_model()`, coupling the supply to actual grid operation. The optimizer now reasons about true network flows (e.g. reserve output as a result of node dispatch), not just supply composition.

Objective Function

$$\min \sum_i \text{penalty} \cdot (a_i + r_i) - \sum_j w_j \cdot v_j^{\text{objective}}$$

Minimize supply perturbation while maximizing the target operating flow (e.g. reserve export `EX_res(e)`).

Pipeline: Sources → Flows → Constraints

Site supply (units/day)

$$v_i = D_i \times (1 - A_i) \times 1000 / (Mw_i \times B_{total})$$

Source flows (kW/h/MVA)

$$net.medium = medium[...]$$

Upper bounds on import lines in cooperative_tradeoff

! TO DO — Supply Toolbox

This section is missing a presentation of the **supply toolbox** and how it works:

- What is the supply toolbox and what problem does it solve?
- How does it convert raw source/power data into transfer flows compatible with GridFlow?
- What databases or source catalog tables does it rely on?
- How does it handle loss rates A_i and conversion factors Mw_i ?
- How does it interface with the supply optimizer and the cooperative tradeoff pipeline?

This must be documented before this chapter can be considered complete.

1.d Minimal Supply

Computing the minimum source set required to sustain network dispatch.

1.e Supply Optimization

Identifying supply profiles that maximize a target operating output for the network.

Appendix — Code Reference

A.1 Optimization Engine

Each section maps an optimization concept to the exact lines of GridFlow source code that implement it.

Base constraints ($S \cdot v = 0$, $\text{objective_}[nd] \geq 0$, transfer bounds, **network_objective** equality) are all added in Step 1 inside `network.py : Network.__init__()`. **Optimization-specific constraints** (individual min dispatch floor, network dispatch bounds) are added in Step 2 inside `problems.py : cooperative_tradeoff()` and `util.py : _apply_min_dispatch()`.

Step 1 — Problem Construction

Lines $\rightarrow$ flow variables + $S \cdot v = 0$

```
# network.py : 262-265, 277
for r in model.lines:
    r.id = r.global_id + suffix          # unique namespace per node: "LNK_nd1"
    r.network_id = idx
self.add_lines(model.lines)             # GridCore creates v_forward, v_reverse >= 0
                                         # and encodes  $S \cdot v = 0$  for every bus
```

objective_[nd] constraint

```
# network.py : 278-286
o = self.solver.interface.Objective.clone(model.objective, model=self.solver)
obj += o.expression * row.capacity      # accumulates for network_objective
node_obj = self.problem.Constraint(
    o.expression, name="objective_" + idx, lb=0.0  # lb=0  $\rightarrow$  dispatch_nd >= 0
)
self.add_cons_vars([node_obj])
```

network_objective variable + equality constraint

```
# util.py : 205-210 / network.py : 297-298
def add_var_from_expression(model, name, expr, lb=None, ub=None):
    var = model.problem.Variable(name, lb=lb, ub=ub)
    const = model.problem.Constraint(
        (var - expr).expand(), lb=0, ub=0, name=name + "_equality"
    ) # network_objective =  $\sum$  capacity_nd  $\times$  dispatch_nd
    model.add_cons_vars([var, const])
    return var

net_obj = add_var_from_expression(self, "network_objective", obj, lb=0)
self.objective = self.problem.Objective(net_obj, direction="max")
```

Transfer flow bounds + capacity coupling

```
# network.py : 323-385 (__add_transfers)
export = len(r.sinks) == 1
lb = r.lower_bound / self.mass if export else -r.upper_bound / self.mass

coef = info.capacity
r.add_buses({medium_bus: coef if export else -coef}) # capacity as incidence coef
```

Solver selection and Gurobi config

```
# network.py : 138-143
solver = [s for s in ["cplex", "gurobi", "hybrid", "glpk"]
          if s in gridcore.util.solver.solvers][0]

# util.py : 298-307
if interface == "gurobi":
    solver.configuration.lp_method = "barrier"
    solver.problem.Params.BarConvTol = 1e-9
    solver.problem.Params.CrossoverBasis = 1
    solver.problem.Params.NumericFocus = 2
    solver.problem.Params.Threads = 1
```

Step 2 — Cooperative Tradeoff

Main function

```
# problems.py : 70-100
def cooperative_tradeoff(network, min_dispatch, fraction, flows, pfba, atol, rtol):
    with network as net:
        _apply_min_dispatch(network, min_dispatch)      # optional: objective_[nd].lb = min_dispatch

        # Phase 1
        net.objective = net.scale * net.variables.network_objective
        min_dispatch = optimize_with_retry(net, ...) / net.scale

        # Phase 2
        regularize_l2_norm(net, 0.0)
        for fr in fraction:
            net.variables.network_objective.lb = fr * min_dispatch  # dispatch floor
            net.variables.network_objective.ub = min_dispatch        # dispatch ceiling
            sol = solve(network, flows=flows, pfba=pfba, ...)
            if not pfba and sol.status != OPTIMAL:
                sol = crossover(net, sol, flows=flows)
```

Phase 1 — LP objective + solver call

```
# problems.py : 78 / solution.py : 214-223
net.objective = net.scale * net.variables.network_objective  # maximize

def optimize_with_retry(net, message):
    sol = net.optimize()      # first solver call (LP)
    if sol is None:
        network.solver.problem.reset()  # Gurobi reset on failure
        sol = net.optimize()
    return sol.objective_value
```

Phase 1 — Optional individual min dispatch

```
# util.py : 274-283
for nd in network.nodes:
    obj = network.constraints["objective_" + nd]
    obj.lb = (1.0 - rtol) * min_dispatch[nd] - atol
```

```

if obj.lb < atol:
    obj.lb = 0

```

Phase 2 — L2 objective construction (LP → QP)

```

# problems.py : 53-66
def regularize_l2_norm(network, min_dispatch):
    l2 = Zero
    for nd in network.nodes:
        node_obj = network.constraints["objective_" + nd]
        ex = sum(v for v in node_obj.variables if (v.ub - v.lb) > 1e-6)
        if not isinstance(ex, int):
            l2 += (network.scale * (ex**2)).expand() # quadratic term per node
    network.objective = -l2 # minimize L2 maximize -L2

```

Phase 2 — Solver call + reading the solution

```

# solution.py : 177-196, 103-119
def solve(network, ...):
    network.solver.optimize() # QP solver call (Gurobi)
    if pfba and flows:
        add_pfba_objective(network, atol, rtol)
        network.solver.optimize()
    return NetworkSolution(network)

# reading dispatch rates from primal values
gcs[nd] = network.constraints["objective_" + nd].primal
if interface == "gurobi":
    gcs = gcs.abs() # Gurobi/optlang bug workaround

```

Phase 2 — Crossover fallback (if QP returns non-optimal)

```

# solution.py : 237-251
net.variables.network_objective.ub = net_dispatch + 1e-6
net.objective = net.scale * net.variables.network_objective # back to LP
for nd in net.nodes:
    net.constraints["objective_" + nd].ub = max(const.lb, gcs[nd]) # fix QP dispatch rates

```


A.2 Supply Math to Code

Part 1 — Supply as a Constraint (`cooperative_tradeoff`)

Import flow upper bound

$$0 \leq v_{\text{import},i} \leq v_i$$

where v_i is derived from the site's raw supply:

$$v_i = \frac{D_i \times (1 - A_i) \times 1000}{Mw_i \times B_{\text{total}}}$$

→ `grow.py:55` (`net.medium = medium[medium.index.isin(ex_ids)]`)

Active in both steps of `cooperative_tradeoff`:

- Step 1 (LP): limits the feasible space for maximizing network dispatch
- Step 2 (QP): same bounds apply when redistributing flows under the tradeoff constraint

Part 2 — Supply as a Decision Variable (`supply_optimizer.py`)

Decision Variables

f_i — Amount of source i (units)

$$f_i \in [f_i^{\min}, f_i^{\max}]$$

`min_amt = current_amt × (1 - max_source_change_pct / 100)`

`max_amt = current_amt × (1 + max_source_change_pct / 100)`

For new candidate sources: $f_i \in [0, 200u]$

→ `supply_optimizer.py:457` (existing sources) · `supply_optimizer.py:468` (new candidate sources)

a_i — Addition relative to current profile

$$a_i \in [0, \max(0, f_i^{\max} - f_i^{\text{current}})]$$

If $f_i^{\max} < f_i^{\text{current}}$, the upper bound is clamped to 0 and $a_i = 0$.

→ `supply_optimizer.py:460`

r_i — **Reduction relative to current profile**

$$r_i \in [0, \max(0, f_i^{\text{current}} - f_i^{\text{min}})]$$

If $f_i^{\text{current}} < f_i^{\text{min}}$, the upper bound is clamped to 0 and $r_i = 0$.

→ `supply_optimizer.py:461`

$\delta_i \in \{0, 1\}$ — **Was source i modified?** Binary indicator variable.

→ `supply_optimizer.py:480`

Constraints

Source balance

$$f_i = f_i^{\text{current}} + a_i - r_i$$

→ `supply_optimizer.py:475`

Max sources changed

$$\sum_i \delta_i \leq N$$

→ `supply_optimizer.py:488`

Change indicator (big-M)

$$a_i + r_i \leq 1000 \cdot \delta_i$$

→ `supply_optimizer.py:484` (existing) · `supply_optimizer.py:486` (new candidates)

Load constraints

$$\text{load}^{\text{min}} \leq \sum_i \frac{\text{load}_i}{100} \cdot f_i \leq \text{load}^{\text{max}}$$

→ `supply_optimizer.py:991-1004` · `supply_optimizer.py:971` (`_add_load_constraints()`) · `supply_optimizer.py:491`

FBA — Line flow bounds (*optional, requires `--model`*)

$$v_r^{\text{min}} \leq v_r \leq v_r^{\text{max}} \quad \forall r$$

→ `supply_optimizer.py:1103`

FBA — Incidence steady-state (*optional, requires `--model`*)

$$\sum_r S_{m,r} \cdot v_r = 0 \quad \forall m$$

→ `supply_optimizer.py:1107-1113` (inside `_embed_network_model()`)

FBA — Supply-transfer linking (*optional, requires `--model`*)

$$v_{EX_i} + \frac{\text{supply_flux}_i}{\text{FLUX_CONVERSION_FACTOR}} \geq 0$$

Same structure as Part 1, but v_i^{supply} is a linear expression in f_i rather than a fixed value.

→ `supply_optimizer.py:1119-1131` · `supply_optimizer.py:500`

Objective Function

$$\min \underbrace{\sum_i \text{penalty} \cdot (a_i + r_i)}_{\text{minimize perturbation}} + \underbrace{\sum_j w_j \cdot \text{obj_term}_j}_{\text{operating objective}}$$

obj_term routing:

- With FBA model → `obj_term` = FBA flow variable v_{EX_j}
- Without model → `obj_term` = supply flow expression $\sum_i \text{coeff}_{ij} \cdot f_i$

→ `supply_optimizer.py:507-513` (perturbation penalty) → `supply_optimizer.py:515-542` (operating objective, handles `RES_total`) → `supply_optimizer.py:544` (`gmodel.setObjective(obj_expr, GRB.MINIMIZE)`) → `supply_optimizer.py:1037` (`_add_objective_term()`) → `supply_optimizer.py:1051-1058` (FBA path) · `supply_optimizer.py:1063-1068` (supply flow fallback)

Supply Flow Expressions

$$\text{flow}_m = \sum_i \frac{g_{m,i}}{100} \cdot \frac{1000}{Mw_m} \cdot f_i$$

→ `supply_optimizer.py:1006` (`_build_flow_expressions()`) → `supply_optimizer.py:1028` (coefficient computation) → `supply_optimizer.py:494` (called from `_solve_with_gurobi`)

Assessing the validity of current core engine

Validation Using Measurement Datasets

Proposed plan

Here is the plan as I see it — let me know if it looks OK:

- INTRO:
 - Reminder of why it is not obvious to find datasets and carry out this validation
 - Which results exactly interest us, in the absolute? (which flows, the dispatch rates of which nodes?)
 - Reminder that what interests us is not necessarily an absolute error per sample, but rather R^2 values at the scale of a whole fleet
- ALREADY DONE: an experiment presented at the steering committee
 - Presentation of the study context
 - Presentation of the results obtained
 - Link to the study and the results in the application?
- PLAN
 - Will there be internal partner data available a priori? Will it be hard to obtain? (I imagine so)
 - Which open-source datasets are possible?

Sensitivity Analysis

How model outputs respond to perturbations in key parameters (capacities, supply, tradeoff).

Robustness Plan for Calls to Optimization Solver

Handling solver failures, numerical instabilities, and edge cases in production.

Securing IP while improving current engine

Using training to correct errors Other propositions?

Long-term R&D vision

R&D use cases assessment

Proposal from the product team

From memory, it seems there is at least:

- Edge substation modeling
- From legacy- to Voltgrid-inspired grid health score
- Longitudinal analysis
- Modeling interaction with substation and transmission layer

Sponsors and KOL strategy

Proposal from the product team

To be filled in tomorrow with the engine team. Goal: first presentation to the steering board in CTM on 15 March 2027.

We need to:

- map the internal sponsors of Voltgrid,
- recall the stakeholders already identified
- propose partnership actions

Change and integration

Usage ramp-up

Plan for progressively rolling out the engine to operators and scaling the number of concurrent network simulations.

Datasets

Inventory of the measurement datasets used to calibrate and validate the network model, with their sources and access constraints.